Sojern × Brew: embed hotel-branded email creation

Technical implementation brief · 22 October 2026 · SDK baseline: `@brew.new/sdk@13.2.0`

Use the live Copy controls or Markdown download for code and complete agent briefs. The PDF is the portable reading handoff.

Try the headless integration lab

Open https://sojern-brew-demo.vercel.app. Enter your own Brew API key in the connection form. No Brew login or fixed demo organization is used. The app discovers brands and emails within that key's authority, including brand-scoped keys. Organization keys with `brands` and `emails` permissions can extract new brands; existing account capacity and credit limits apply.

The key stays in page memory until disconnect or reload. The app sends it over HTTPS only to its stateless same-origin backend, which calls the official SDK. It does not store the key in cookies, browser storage, a database, logs, downloaded files, or the canvas iframe. This user-supplied-key playground differs from a production staff app: keep organization keys in your own backend secret settings for the production integration described below.

- Connect your key, then select an existing brand or extract a public HTTPS website. The new brand ID is retained before polling.
- Open a group or email from the nested picker. For a locale variant, select the template in the canvas, choose **Use selected** in Create, choose the language, and enter the subject in that language. Generation returns a separate email and version. Clear the template field for a fresh design.
- Apply an AI edit to create a new version, or click the embedded email to edit it manually. Editing is enabled by default for API-key canvas sessions. Create and edit requests retain the group iframe.
- Use **Save & HTML** to flush manual edits and retrieve the exact saved version. Reopen to verify persistence. No email is sent.

Rename a named group from its submenu, or use **Rename email** for the current design. Ungrouped is reserved.

Run or adapt the example

The page, guide, prompts and complete source ZIP are public. No Vercel account or Brew login is needed to read or download them. Account actions require your own Brew API key.

The example uses strict TypeScript, SDK 13.2, a static browser bundle, a small React/Radix picker and one stateless Vercel function. Its UI and API response types reuse the public SDK contracts. It adds no database or login system.

The maintained source lives in GetBrew/sojern-brew-demo. The public source ZIP is also available.

Download and unpack the source, or clone `GetBrew/sojern-brew-demo`. Use Node 24, then:

```
npm ci --ignore-scripts
npm run check
cp .env.example .env.local
# Set DEMO_SESSION_SECRET in .env.local; generate with openssl rand -hex 32.
# DEMO_ORIGIN defaults to http://localhost:4400 in the example file.
npm run dev
```

Open http://localhost:4400 and enter your key in the connection form. For Vercel, use `npm run build`, output `public`, Node 24, and set `DEMO_ORIGIN` to the exact HTTPS domain plus `DEMO_SESSION_SECRET` in runtime settings. Disable deployment login protection for the intended public demo domain. Never configure an organization API key as a fallback.

`npm run check` runs formatting, strict types, isolated API/security tests and the browser build. `npm run build` also checks types and verifies that the included PDF matches this Markdown. Start with `web/app.ts`, `api/demo.ts` and the source README. The live implementation tab explains the architecture and provides copyable SDK examples; the agent tab contains this guide's complete implementation briefs.

Optional publishable canvas client ID

Open **Settings → API → Client apps** in Brew. An organization admin can choose **All brands** for every current and future hotel brand, or select one specific brand. Brand members can create IDs only for their authorized brands. Allow `https://sojern-brew-demo.vercel.app`, enable editing, then enter the `brew_client_…` ID in the connection form and choose **Apply**. The demo keeps that ID in connection memory when you change brands, and passes the selected `brandId` to the SDK. Brew enforces the configured scope. A single-brand ID rejects other brands; apply a matching ID or clear it to return to API-key sessions. Changing accounts, disconnecting or reloading clears the ID. Leaving it blank requires no client app setup.

A client ID grants canvas access only. It cannot extract brands, generate emails or perform AI edits. Its allowed brand scope and exact origins control canvas access. Each session stays bound to one brand and one email/group, even with an organization-wide ID. Changing the client app’s scope invalidates existing sessions. Use the same client-ID pattern in your own app:

```
import { createEmailCanvas } from "@brew.new/sdk/embed";

const canvas = createEmailCanvas(container, {
  clientId: configuredClientId,
  brandId: selectedBrandId, // required for All brands IDs, including ungrouped
  emailId: selectedEmailId,
  onSaved: (receipt) => retainSavedVersion(receipt),
});
await canvas.flush(); // before navigation, AI editing or HTML retrieval
canvas.destroy();
```

The SDK renews and cleans up its sessions. A publishable client ID does not authenticate individual staff; use the advanced backend-issued grants below when per-user or per-email authorization is needed.

What Sojern can build

Keep Sojern’s existing internal application, staff login, hotel records, and sending system. Add Brew’s brand extraction and email APIs to the backend, then embed Brew’s hosted email canvas in the designer’s workspace. Staff do not need a separate Brew login.

The workflow is: onboard a hotel website → review its extracted brand → generate an email → refine it with AI → make manual design edits → retrieve the saved HTML for the existing delivery workflow.

This integration is independent of Brew’s implementation backend. Sojern does not need Convex, a Brew dashboard session for designers, a new database, or a new queue system.

Demo sequence for the technical team

Use the live workspace for the walkthrough and the Implementation tab for the code. Allow a few minutes for extraction and each AI operation. All actions use the connected account's real quota and saved data. Use a dedicated demo hotel and group.

Step	In the demo	Explain to Sojern
1. Connect	Enter an organization API key. Leave client ID blank for the editable session-provider path.	The playground accepts a key in memory; Sojern's staff app keeps it on the backend behind existing SSO.
2. Onboard a hotel	Select a ready hotel, or extract a new public HTTPS hotel website when a brand slot is available. Inspect Brand context.	Save hotel → brandId immediately. Extraction continues if the polling request ends.
3. Manage brand facts	Expand Read & update brand information. Review approved identity fields before saving.	Brand context is readable and writable programmatically. Design guidance saves replace the full Markdown document.
4. Create a template	In the group/email picker choose New group. Open Create, clear any template, use an English subject, and create an email.	Generation returns emailId and emailVersionId. The hotel mapping stays on Sojern's server.
5. Localize it	Select the English email in the canvas, choose Use selected in Create, choose French, and enter `Votre prochain séjour` as the subject. Create the variant.	The callback exposes the displayed email/version. referenceEmailId creates a separate email from the source's latest saved content; AI localization needs review.
6. Browse and rename	Open the nested group/email menu. Rename the group in its submenu and the current email with Rename email.	These are metadata updates, without an AI run or new content version.
7. Refine	Select the French variant, open Refine and shorten its introduction. Then click its text in the canvas and make a manual change. Pan and Ctrl/pinch zoom while the pointer is over the email.	An AI edit creates a version. Manual editing is enabled; the canvas uses Brew's shared Rust/WASM renderer. The iframe persists across create/edit results.
8. Save and prove it	Choose Save & HTML, inspect the exact version, close the dialog and Reopen. Switch v1/latest to show the callback, then disconnect.	flush confirms manual saves; read the exact version, including history. Reopen deliberately creates a new session. No email is sent.

Suggested fresh-design brief:

Create an on-brand hotel newsletter for opted-in guests. Use approved brand facts and assets, one hero and one clear CTA. Do not invent rates, dates, discounts, availability or amenities. Design only; do not send.

Suggested locale brief:

Create a French (fr-FR) version of the selected reference. Preserve its layout, section order, images, URLs, hotel names and approved facts. Translate visible copy, preview text and CTA labels. Do not add offers or send mail.

Choose the subject in the target language explicitly; it is a separate API field. A remix uses AI and may vary the design. For an exact copy, use `emails.clone`; for an exact historical reference, clone that version before remixing. The demo's Use selected action references latest and does not perform that historical clone for you.

When a live action cannot finish

- **No spare brand slot / 402:** use a previously extracted hotel for the remaining demo; arrange account capacity before promising fresh onboarding. Do not delete another hotel's brand.
- **Existing domain / 409:** select the existing brand only when it belongs to the authorized hotel.

- **Extraction still running:** keep the returned brandId and choose Refresh live state. Do not create another brand to retry polling.
- **AI timeout:** read saved state and versions. Reuse a create intent for generation; reconcile AI edits before intentionally repeating a paid PATCH.
- **Canvas cannot edit:** check key permissions, an outstanding editor lease, and client-app editing/origin configuration. The client ID must allow the selected brand; All brands reaches current and future brands in its organization.
- **GPU initialization failure:** handle onError, retry in a supported browser and retain saved IDs. WebGPU or WebGL2 is required; hardware performance must be evaluated on Sojern's devices.

The smallest architecture

```
Sojern internal app — existing staff SSO and hotel authorization
    │
    ├── Sojern backend ── server-only organization API key ── Brew REST / SDK
    │       ├── Hotel record → Brew brandId
    │       ├── Email record → hotel, emailId, version IDs
    │       └── Canvas grant → staff owner, target, permissions, sessionId
    │
    └── Brew SDK canvas ── short-lived server-authorized grant ── Brew iframe
            └── Save → await flush() → Sojern backend → exact-version HTML
```

Sojern owns staff access, the mapping of hotel records to Brew brands, and delivery approval. Brew hosts the brand extraction, email generation, saved designs, and embedded editor. Keep authorization on Sojern’s backend on every operation.

Capability and boundary map

Need	Public API / SDK	Notes
Extract a new hotel brand	`POST /v1/brands`; `platform.brands.create()`	Organization key with `brands`; asynchronous; available brand slot and non-empty credit balance required.
Discover permitted brands	`GET /v1/brands`; `platform.brands.list()`	Paginated. Organization key sees its organization; a brand key sees its bound brand.
Read extraction status	`GET /v1/brands/{brandId}`; `brands.get({ brandId })`	Wait for `ready: true`; use `brands.waitForExtraction()` on the backend.
Read brand information	`GET /v1/brand?include=identity,emailDesign,imageStyle,logos`; `hotel.brand.get()`	Select hotel with `withBrand(brandId)` for an organization key.
Update brand information	`PATCH /v1/brand`; `hotel.brand.patch()`	`identity` shallow-merges; `emailDesign` and `imageStyle` replace their entire Markdown documents. Requires `emails`.
Generate an email	`POST /v1/emails`; `hotel.emails.generate()`	Synchronous HTTP result after generation; may return prose instead of design IDs.
Browse and name groups	`hotel.emailGroups.list/create/update()`	Paginate groups/emails. Ungrouped is reserved; metadata renames do not run AI.
Copy an exact template	`hotel.emails.clone({ emailId, emailVersionId, groupId, title })`	Creates another email; omit version only when latest is intended.
Create locale variants	`hotel.emails.generate({ referenceEmailId, groupId, prompt, subjectLine })`	AI remix from latest; save a separate email per locale and review translation.
AI-edit an email	`PATCH /v1/emails/{emailId}`; `hotel.emails.edit()`	Prompt edits create a version; optionally pin `emailVersionId` as the baseline.
Rename / update subject	Same PATCH; `title` / `subjectLine`	Metadata-only edits are deterministic and do not start AI generation.
Embed designer access	`POST /v1/canvas/sessions`; `hotel.canvas.sessions.create()`	Bound to an authorized target, user, exact parent origin, and permissions.
Renew / revoke access	`canvas.sessions.refresh(sessionId)` / `.revoke(sessionId)`	Reauthorize ownership on the Sojern backend; same issuing API principal.
Retrieve saved HTML	`GET /v1/emails/{emailId}?include=html&emailVersionId=…`	Read the exact version reported by the canvas save event.
Upload brand assets	`hotel.content.uploadImage()`	Separate from replacing brand logos; the canvas also supports designer uploads.

A brand-scoped API exists for reading and updating brand context and operating on its emails. Creating another brand is an organization operation. `withBrand()` selects a brand; it does not turn an organization key into a restricted credential. Never accept a browser-provided brand ID as authorization.

Current brand identity PATCH fields are `brandName`, `description`, `tagline`, `contactEmail`, `contactPhone`, and `address`. GET may return additional extracted facts. Do not assume every returned field is writable.

The public contract currently has no brand deletion endpoint, no restart-extraction endpoint for an existing brand, and no public logo-replacement operation. The canvas is a Brew-hosted iframe; the SDK offers no custom theme, logo, or custom-domain options. Confirm these boundaries during technical evaluation if they are purchasing requirements. Brand slots and AI capacity must match the hotel-brand portfolio.

Brand domains are unique within an organization. Map each hotel brand to its saved brandId; properties sharing an authorized brand/domain may reuse that mapping. Keep property-specific facts and staff access in Sojern's existing records. A domain conflict alone never proves that a caller may use an existing brand.

All required authoring operations can use the organization API key and backend-issued canvas grants. The publishable client app is optional; designers need no Brew dashboard session.

Implementation breakdown

Task	Smallest deliverable	Completion evidence
1. Hotel onboarding	One server client, existing hotel → brand mapping and extraction/status routes; authorized brand context reads/updates.	Readiness, retry intent and cross-hotel checks pass.
2. Email authoring	Generate/edit, locale remixes, exact cloning, paginated groups/emails and metadata renames.	New locale and edit receipts; source unchanged; renames read back.
3. Designer workspace	Scoped grants and one persistent editable group canvas inside the existing UI.	Selection/history callbacks, save/HTML and pan/zoom work without remounting on writes.
4. Pilot handoff	Deployed origin/permissions/storage checks and this guide with copyable prompts.	Evidence identifies the deployed revision, actual checks and remaining configuration gaps.

Use the existing authentication, database and runtime. An additional queue is needed only if the current host cannot wait for generation and has no existing worker path.

Backend implementation

Install the SDK in the existing application:

```
npm install @brew.new/sdk@^13.2.0
```

Create one server-only client. Use an organization key with `brands` and `emails` permissions. The API base is `https://brew.new/api`; REST paths below append `/v1/…`. The SDK defaults to that production base.

```
import { createBrewClient } from "@brew.new/sdk";

export const platform = createBrewClient({
  apiKey: process.env.BREW_API_KEY!,
});
```

Use the existing secret manager. Never put the organization key in a browser bundle, a `NEXT_PUBLIC_*` variable, an iframe URL, or an agent prompt. Initial API-key issuance is an authenticated admin task; a key cannot mint other API keys through the API.

1. Onboard a hotel

Authorize staff to onboard that hotel, then create a brand:

```
const created = await platform.brands.create(
  { url: hotel.websiteUrl },
  { idempotencyKey: hotel.onboardingRequestId },
);
// Persist created.brandId on the existing hotel record immediately.
const status = await platform.brands.waitForExtraction(created.brandId, {
  maxWaitMs: 300_000,
});
if (!status.ready) throw new Error(status.error ?? `Brand is ${status.status}`);
const hotelBrew = platform.withBrand(created.brandId);
```

`hotel` above must come from Sojern's authorized server records. Keep its brand ID even if polling times out. Extraction continues; poll that ID again rather than creating another brand. For a short-lived server request, return the saved ID and expose a status route through the existing app. No queue is required solely to poll extraction.

Use a new idempotency key per intentional create action and preserve it for retries. A matching domain in the organization produces `409 BRAND_DOMAIN_CONFLICT`; reuse it only if Sojern's records authorize that hotel to use the existing brand.

2. Read and manage its brand context

```
const context = await hotelBrew.brand.get({
  include: ["identity", "emailDesign", "imageStyle", "logos"],
});

await hotelBrew.brand.patch({
  identity: { tagline: "Your approved hotel tagline" },
});
```

Present extracted facts for review. Keep a copy or revision record in Sojern's existing audit system before replacing design guidance. To change `emailDesign` or `imageStyle`, read, edit, and send the complete intended Markdown document; partial text replaces the previous document.

3. Generate and AI-edit

```
const generated = await hotelBrew.emails.generate(
  {
    prompt:
      "Create an on-brand hotel newsletter for opted-in guests. Use the approved hotel facts.
One clear CTA. Do not invent rates, availability, discounts, or amenities.",
    subjectLine: "Plan your next city escape",
    category: "newsletter",
  },
  { idempotencyKey: emailRequestId },
);
if (!("emailId" in generated)) throw new Error(generated.response);
// Persist emailId, emailVersionId and hotel ownership before returning IDs.

const edited = await hotelBrew.emails.edit({
  emailId: generated.emailId,
  emailVersionId: generated.emailVersionId,
  prompt:
    "Shorten the introduction and change the primary CTA to Discover your stay.",
});
if (!("emailId" in edited)) throw new Error(edited.response);
```

`emailRequestId` is a stable ID for this generation action in Sojern's existing request model. The SDK waits for generation/edit completion with a default four-minute request timeout for these operations. Allow enough runtime in the backend; use an existing worker if the host's limit is shorter.

PATCH AI edits have no replay protection. After an ambiguous timeout, inspect versions and any running generation before deciding to retry. `409 EMAIL_RESULT_UNAVAILABLE` with `details.committed: true` means a design was saved: persist the returned `emailId`, `emailVersionId`, and `runId` against the authorized hotel, then read that exact version. Do not repeat the paid prompt to recover the response.

Templates, locales and groups

SDK 13.1.0 adds `onEmailSelectionChange`. Keep a group canvas mounted and record its resolved displayed selection:

```
const [selected, setSelected] = useState<CanvasSelectedEmail | null>(null)

<EmailCanvas
  ref={canvas}
  clientId={clientId}
  brandId={brandId}
  groupId={groupId}
  onEmailSelectionChange={(emails) => setSelected(emails[0] ?? null)}
/>
```

Import `CanvasSelectedEmail` from `@brew.new/sdk/embed`.

Field	Meaning
`emailId`	Logical email ID shared by its versions
`emailVersion`	Displayed version: a positive number or `"latest"`
`emailVersionId`	Exact saved row ID, or `null` for a legacy row

The callback receives `[]` on deselection or removal and while a version is switching or unresolved. The existing `onSelectionChange` still returns IDs.

After authorizing the selection on your backend, create a new locale email:

```
const localized = await hotelBrew.emails.generate(
  {
    referenceEmailId: selected.emailId,
    groupId: ownedGroupId,
    subjectLine: "Votre prochain séjour",
    prompt:
      "Create this hotel email in French (fr-FR). Preserve its layout, images, URLs and
approved facts. Translate the copy, preview text and CTA. Do not send.",
  },
  { idempotencyKey: localeRequestId },
);
```

This creates a separate email. Keep the source and each locale's returned email/version IDs in your existing records. Review the generated translation and layout before approval.

`referenceEmailId` reads the reference's latest saved version. To remix an exact historical selection, first call `emails.clone({ emailId, emailVersionId, title })`.

Use the clone's `emailId` as the reference. A clone copies content exactly; remixing runs AI.

Keep the canvas mounted during writes

Generate into the mounted group's `groupId`. New emails and edited versions arrive through its live subscriptions. Avoid a React `key` based on the current email or version ID.

Before an API AI edit, await `canvas.current.clearSelection()`. It saves pending changes and releases the editing lease without replacing the iframe. Prevent new manual selection while your backend request runs.

Capture the requested email/version before awaiting flush or clearSelection. Save and deselection callbacks may update the current UI selection. Use that captured exact `emailVersionId` as the edit baseline, then call `focusEmail(emailId)` after the updated email is available. If saving fails, keep the editor open and do not start the API write.

`clearSelection()` is also available on `createEmailCanvas` controllers. Changing the authorized brand or group still requires a new scoped canvas.

Rename groups and emails

```
await hotelBrew.emailGroups.update({
  groupId: ownedGroupId,
  name: "Welcome locales",
});
await hotelBrew.emails.edit({
  emailId: ownedEmailId,
  title: "Welcome · French",
});
```

These metadata-only updates do not run AI or create versions. Ungrouped is a reserved group and cannot be renamed.

4. Issue a private canvas grant

Implement one same-origin session endpoint using Sojern's existing staff session and CSRF protection. On every request:

- Authenticate staff and authorize access to the selected hotel and email or group.
- Resolve the Brew brand and email/group IDs from server-owned records.
- Choose `read` or `read, edit, presence` on the server.
- For renewal, confirm the session belongs to this staff member, hotel, email, and issuer before refresh; recheck current access.
- Return the grant as JSON with `Cache-Control: no-store`.

Inside that authorized handler:

```
const session = await hotelBrew.canvas.sessions.create({
  target: { type: "group", groupId: ownedGroup.groupId },
  user: { externalId: `${hotel.id}:${staff.id}`, name: staff.displayName },
  parentOrigin: configuredAdminOrigin,
  permissions: ["read", "edit", "presence"],
});
```

`ownedGroup`, `staff`, and `hotel` are server-authorized records. Use `emailGroups.create()` for a named workspace; generation without groupId belongs to the reserved `ungrouped` group. An email-only editor can use `{ type: 'email', emailId: ownedEmail.emailId }` instead. `configuredAdminOrigin` is a trusted exact HTTPS origin, such as `https://admin.example.com`, without a path or trailing slash. Development accepts loopback HTTP. Store grant ownership in the app's existing data store or session mechanism. The returned bootstrap token is single-use; do not cache it, log it, or put it in a URL. On logout/access removal revoke the issued grant. Custom session providers do not revoke automatically on unmount. When replacing a canvas with a newly issued grant, flush and revoke the old owned grant before mounting the replacement; an abandoned grant can retain its editing lease until that lease expires.

5. Embed, save, and retrieve the exact result

React 18/19 can use `EmailCanvas` from `@brew.new/sdk/react`; other frameworks can use `createEmailCanvas` from `@brew.new/sdk/embed`.

The embedded canvas uses the same GPU renderer as the Brew app. Browsers need WebGPU or WebGL2. Selected emails stay editable while other frames can use GPU previews. Handle `onError` if neither backend can initialize; your app can offer saved HTML while the designer retries.

```
"use client";
import { useRef, useState } from "react";
import { EmailCanvas, type EmailCanvasHandle } from "@brew.new/sdk/react";
import {
  createCanvasSessionProvider,
  type CanvasSelectedEmail,
} from "@brew.new/sdk/embed";

type SavedVersion = { emailId: string; emailVersionId: string };

export function HotelEditor({
  groupId,
  csrfToken,
  onSave,
}: {
  groupId: string;
  csrfToken: string;
  onSave: (saved: SavedVersion) => Promise<void>;
}) {
  const canvas = useRef<EmailCanvasHandle>(null);
  const selected = useRef<CanvasSelectedEmail | null>(null);
  const [error, setError] = useState<string | null>(null);
  const getSession = createCanvasSessionProvider("/api/brew/canvas-session", {
    headers: () => ({ "X-CSRF-Token": csrfToken }),
  });
  async function save() {
    setError(null);
    const pin = selected.current ? { ...selected.current } : null;
    if (!pin?.emailVersionId) {
      setError("Select an email and wait for its exact version to resolve.");
      return;
    }
    try {
      if (!canvas.current) throw new Error("Editor is not ready");
      await canvas.current.flush();
    } catch {
      setError("Save could not complete. Keep this editor open and retry.");
      return;
    }
    try {
      await onSave({
        emailId: pin.emailId,
        emailVersionId: pin.emailVersionId,
      });
    } catch {
      setError(
        "Your edits are saved, but HTML retrieval failed. Retry retrieval.",
      );
    }
  }
  return (
    <>
      <button type="button" onClick={save}>
        Save & HTML
      </button>
      {error && <p role="alert">{error}</p>}
      <EmailCanvas
        ref={canvas}
        groupId={groupId}
        getSession={getSession}
        style={{ height: "80vh" }}
        onEmailSelectionChange={(rows) => {
          selected.current = rows[0] ?? null;
        }}
        onSaved={(saved) => {
          if (selected.current?.emailId === saved.emailId) {
            selected.current = {
```

```
              ...selected.current,
              emailVersionId: saved.emailVersionId ?? null,
            };
          }
        }}
        onError={(error) => setError(error.message)}
      />
    </>
  );
}
```

The group's ID stays stable while the selection and saved versions change. The session endpoint authorizes that group for the hotel; it returns fresh grants on create/renewal. The React adapter uses the latest provider callback without reloading the iframe when the CSRF token changes.

Wire `onDirtyChange` into your existing navigation guard. Flush and revoke the previous owned grant before changing brand/group or leaving the editor. `onSave` calls Sojern's authorized backend with a resolved exact version. If a legacy row has no version ID, require explicit recovery; do not silently substitute latest. Capture the action target before awaiting flush or clearing selection. Save callbacks can update UI defaults; they must not redirect an already requested operation. The save callback is an invalidation signal for matching email/version data, not a reason to remount the canvas.

```
const result = await hotelBrew.emails.get(ownedEmail.emailId, {
  emailVersionId: authorizedVersionId,
  include: "html",
});
if (result.status !== "ready" || !result.html)
  throw new Error("HTML is not ready");
```

Authorize both the email and requested version against the selected hotel. Manual edits can patch a displayed historical version; they do not promote that version to latest. Await `flush()` before navigation, unmounting, AI editing, or HTML retrieval. If the flush fails, retain the editor and show the error. If HTML retrieval fails after a successful flush, tell the designer the edits are saved and offer to retry retrieval.

What Sojern needs

Resource	Minimum requirement
Application	Existing staff SSO, hotel authorization and server routes.
Data	Existing hotel-to-brand and email ownership records; grant ownership for renewal/revocation.
Credentials	Server-side organization API key; `brands` and `emails` scopes.
Runtime	SDK supports Node.js 20+; generation host budget sufficient for its request timeout.
Browser	WebGPU or WebGL2; parent CSP permits `https://brew.new` in `frame-src`; deployed HTTPS admin origin configured exactly.
Capacity	Available brand slots per hotel portfolio, credits for AI actions, route rate-limit handling.
Design control	Approved hotel facts/assets and a designer review step.
Sending	Existing ESP/approval process can consume saved HTML; no sending integration is needed to evaluate the editor.

The current `emails` credential scope also implies `sends` and `domains`. Keep the key server-side and expose only the operations this integration needs. Browser canvas permissions are separate from API-key scopes.

Acceptance criteria for the pilot

- Hotel extraction reaches `ready: true`; errors, timeouts, domain conflicts and plan limits are visible.
- Brand context reads and allowed PATCH fields work; document replacement semantics are understood.
- Generation returns a new email; AI editing returns a new version of the selected email. Both return non-empty exact-version HTML.
- Selection callbacks resolve emailId, displayed emailVersion and exact emailVersionId, and clear stale selection during transitions.
- Locale generation creates a separate email in the mounted group; the reference remains unchanged. Historical references use an exact clone first.
- Paginated group/email browsing and metadata renames persist without creating content versions.
- The iframe and GPU canvas retain DOM identity across generation, AI editing, renames and manual saves. Pan and zoom work over selected email content.
- A designer can manually edit text and styling, flush, reopen, and retrieve the saved result.
- Upload/crop works against actual storage and the resulting image survives save/reopen.
- A viewer cannot edit; unauthorized hotel/email/session selections are denied.
- Renewal checks current staff access; revoked access is denied.
- The deployed HTTPS parent and a denied origin are tested, including CDN/CSP behavior.
- In the production staff integration, no organization API key reaches the browser. The playground deliberately accepts it in page memory, then clears it on disconnect/reload; neither mode writes credentials into logs, prompts or downloads.

Validation record, 22 October 2026

These are evidence boundaries for the demo, separate from the pilot acceptance criteria. Real production browser runs used controlled hotel emails and the official SDK/backend, with no mocked Brew responses.

Area	Observed evidence	Remaining requirement
Hotel onboarding	A fresh hotel extraction completed in the earlier production run. Current discovery/readiness passes; an existing domain returns 409. In the validation account, a fresh extraction attempt now shows 402 BRAND_LIMIT_REACHED and preserves the existing hotel.	Free a brand slot or use a plan with enough capacity before demonstrating another fresh extraction. Ready hotels remain usable.
Authoring and locales	Fresh French reference generation, AI edit, manual save, exact-version HTML, group/email renames, v1/latest callbacks and source preservation passed against Brew main 4748c502. The deployed demo also preserves an explicitly requested historical AI baseline through manual-save callbacks. The iframe stayed mounted through the writes.	Designer review of generated translations, facts and layout.

Area	Observed evidence	Remaining requirement
Renderer and navigation	The real hosted Rust/WASM renderer initializes with WebGL2. Pan/zoom over editable content, exact historical cloning and iframe/GPU persistence through live additions, manual save and history passed.	WebGPU hardware and performance on Sojern's target devices have not been benchmarked.
Brand context and access	Identity PATCH readback preserved all other identity/design values. A different brand cannot read the controlled email (404). Read-only rendering, grant refresh/revoke, missing/invalid key rejection and browser origin/CSP denial passed.	Verify Sojern's own staff/tenant authorization and actual deployment origin.
Image durability	The review found and fixed a shared-picker selection race during crop upload. Production recheck on Brew main 8944ec00 passed authenticated upload, exact-version save, reopen and a real CDN image read. The iframe and GPU canvas stayed mounted during the save; no browser or transport errors occurred.	Repeat with Sojern's own image assets during the pilot.
Credential modes	Editable API-key-issued canvas sessions passed. The demo accepts account credentials in memory and clears them on disconnect/reload.	Organization and specific-brand client-ID scopes pass backend, HTTP and SDK regression tests, including future brands and scope invalidation. A positive production run still needs a real client app configured for the demo origin.

The separate API CI suite stops before writes when its configured TEST_API_URL serves an older commit. This is distinct from the successful targeted production checks; keep the release preflight enabled and update the intended nonproduction test deployment before rerunning that suite.

Prompts for implementation agents

Implement Sojern's hotel-branded email authoring inside our existing application with @brew.new/sdk 13.2.0 or newer. Read this guide and https://docs.brew.new/sdks/typescript/white-label-email-editor first. Reuse existing SSO, hotel authorization, CSRF protection, data store, UI and deployment. Keep organization credentials on the backend and out of prompts, logs and browser bundles. Complete these four tasks in order. Add infrastructure only for a demonstrated need. Report files, configuration, checks and remaining gaps. Use controlled fixtures; do not send mail or delete hotel data.

Copy the paragraph above with all four prompts for one implementation agent, or assign the tasks in order. Each task must preserve the earlier tasks' ownership checks. Configure secrets through your runtime's secret manager.

Prompt 1 - Brand provisioning and management

Add authorized hotel brand provisioning and management with @brew.new/sdk 13.2.0+. Read this guide and https://docs.brew.new/sdks/typescript/white-label-email-editor. Use one server-only organization client with brands and emails permissions and the existing staff/hotel authorization. Persist hotel → brandId immediately after brands.create, retain one idempotency key per onboarding intent, and poll the saved ID with a deadline until ready: true. Preserve that ID across timeouts and failed extraction. Surface slot/credit limits and domain conflicts; never adopt another hotel's brand merely because a domain matches. Add brand context GET and PATCH through withBrand, brand.get and brand.patch. withBrand selects a brand; it does not restrict the credential. Support only writable identity fields; identity shallow-merges, emailDesign/imageStyle replace complete Markdown documents. Do not invent deletion, re-extraction or logo replacement APIs. Reuse existing infrastructure. Verify cross-hotel denial, pagination, readiness, provisioning replay and failure recovery. Report configuration and checks; keep credentials out of output.

Prompt 2 - Templates, locales, groups and AI editing

Add authorized hotel email generation, template localization, AI editing, group/email browsing and metadata renames with @brew.new/sdk 13.2.0+. Resolve brand/email/group ownership from existing server records and use withBrand without mutating a shared client. Require a ready brand; handle design-or-prose result variants and persist returned emailId/emailVersionId before follow-up reads. Preserve an idempotency key for each generation intent. Generate each locale as a separate email in the selected group using an authorized referenceEmailId, target-language prompt and explicitly localized subjectLine. Reference generation reads latest; clone({emailId,emailVersionId,groupId,title}) first for an exact historical reference. Treat localization as AI output requiring designer review, not a guarantee of identical layout. Snapshot the requested email/version before flushing; save callbacks may update UI defaults. Pin that exact emailVersionId for AI edits. PATCH edits lack replay protection: reconcile versions/running work after ambiguity, and recover committed EMAIL_RESULT_UNAVAILABLE receipts without repeating the prompt. Paginate emailGroups.list/emails.list; rename with emailGroups.update({groupId,name}) and emails.edit({emailId,title}), without AI or new versions. Ungrouped is reserved. Reuse existing runtime/worker infrastructure. Verify source preservation, new locale IDs, correct edit baselines, rename readback and cross-hotel/reference denial. Do not send mail.

Prompt 3 - Persistent editable canvas and exact saves

Embed Brew's hosted canvas with @brew.new/sdk 13.2.0+ inside the existing staff app. Reuse SSO, CSRF and hotel authorization. Prefer a same-origin canvas-session endpoint for per-staff/target access: independently resolve authorized hotel, brand, group/email and server-chosen read/edit/presence permissions. Persist session owner/issuer/brand/target; reauthorize create, refresh and revoke. Return uncached grants with the exact trusted HTTPS parentOrigin; never log bootstrap tokens or forward arbitrary session IDs. Use createCanvasSessionProvider with current CSRF headers and EmailCanvas or createEmailCanvas. A publishable clientId is optional when app-wide canvas access on the allowed origin is intended. An organization admin can configure All brands (current and future) or one specific brand; brand members can configure only authorized brands. Pass the selected brandId with the group/email target using SDK 13.2+. Existing brand-bound IDs remain compatible. Keep the ID at connection scope so one All brands ID works when switching hotels or adding brands, while each session remains bound to its chosen brand and target. Scope changes invalidate existing sessions. Client IDs do not authenticate staff or grant generation/brand-management access. Enable editing in the client app. Keep one group canvas mounted across generation, AI editing, renames and manual saves. Never key it by email/version. Retain onEmailSelectionChange emailId, emailVersion and nullable exact emailVersionId; clear stale pins on [] and unresolved transitions. Match onSaved receipts to the selected email. Await flush before navigation/export; if it fails, keep the editor open and block completion. Snapshot the requested email/version before any await so save/deselection callbacks cannot change the action target. Before API AI edits await clearSelection to save and release the editing lease, block new manual selection until completion and focus the result afterward. Reauthorize exact-version HTML reads; never silently replace an unknown historical pin with latest. Report HTML retrieval failure after successful flush as already saved. Use the same hosted Rust/WASM GPU canvas as Brew; require WebGPU or WebGL2 and handle onError. Verify wheel/pinch navigation over email content, editing by default, lease handoff, exact history, iframe/GPU DOM identity, renewal, revocation and cleanup. Custom providers require explicit revocation.

Prompt 4 - Deployed pilot verification and handoff

Review this guide, the actual code and deployment configuration. Run the example source checks (formatting, strict TypeScript, tests and build) and verify anonymous guide, PDF, Markdown and source access. Then validate browser → our backend → Brew API → saved data → hosted canvas with controlled demo records. Record exact fixture email/version IDs before every content write; never choose a newest/recent email for mutation. Use approved credentials only through secret settings and preserve request intent IDs; do not blindly retry paid actions. Check extraction readiness and visible quota/domain errors, brand context update semantics, fresh creation, referenceEmailId locale generation with a localized subject, AI version creation, manual text/style edit, flush/reopen and exact-version HTML. Prove template preservation, selection callbacks across v1/latest, paginated nested group/email browsing, metadata renames and iframe/GPU persistence across writes. Check pan and zoom over selected email content and default editing. Exercise upload/crop durability, viewer permissions, wrong-hotel/email/reference/session denial, allowed/denied origins, CSRF, renewal and revocation. Test optional clientId mode separately from API-key sessions: one All brands ID across current and newly added brands, a specific-brand ID rejecting other brands, foreign-organization denial, and invalidation after narrowing or broadening scope. Use a real configured ID for the deployed-origin check. Inspect browser console/network and downloaded PDF/Markdown/source for secrets and consistency. Reuse existing infrastructure. Report deployed revision, actual evidence, and each untested path or account/configuration blocker. Do not claim hardware performance from a software GPU check. Do not send mail or delete existing hotel data.

Canonical resources

- Full Sojern demo source (standalone GetBrew repository): https://github.com/GetBrew/sojern-brew-demo

- White-label implementation plan and copyable agent prompts: https://docs.brew.new/sdks/typescript/white-label-email-editor
- Canvas API, React and framework-neutral integration: https://docs.brew.new/sdks/typescript/embedded-email-canvas
- Organization versus brand credentials: https://docs.brew.new/api-reference/api/organization-and-brand-scope
- Authentication and permission scopes: https://docs.brew.new/api-reference/api/authentication
- Credits and capacity: https://docs.brew.new/api-reference/api/credits
- Machine-readable live API contract: https://brew.new/openapi.json
- Machine-readable API guidance: https://brew.new/api/v1/llms.txt
- Open-source SDK and embedded demo: https://github.com/GetBrew/typescript-sdk/tree/main/examples/embedded-email-canvas

This guide is the implementation and demo handoff. The validation table above distinguishes exercised production paths from remaining pilot checks.